

PART I - FOUNDATION PLC PROJECTS

Project 9 - Elevator Control System

Version 1 - Foundation PLC Project

PLC Platform: Allen-Bradley MicroLogix 1100

Programming Software: RSLogix Micro Starter Lite 8.30

Prepared By: Sharmin Rahman

1. Project Overview

The Elevator Control System was developed to introduce state-based motion control, request handling, direction control, door sequencing, and timer-simulated travel in a two-floor elevator application.

The current Version 1 screenshots show Floor 1 and Floor 2 position memory, a stored request condition, Move Up and Move Down commands, door-open state, motor direction outputs, a running indicator, travel timers, and a door-open timer.

The fully visible request path begins with a Floor 2 button while the elevator is stored at Floor 1. The program accepts the request, checks the Door Closed condition, moves upward for five seconds, updates the stored floor position to Floor 2, opens the door for five seconds, then clears the request and returns to an idle state.

The screenshots also contain the complete downward-motion, Floor 1 arrival, and Floor 1 door-open logic. However, the separate rung that creates a new Request_Active condition for a Floor 1 request is not visible. I document the visible down sequence exactly as implemented without inventing the missing request-generation logic.

2. Engineering Objective

The objective of this project was to develop and verify a PLC program that coordinated elevator requests, floor-state memory, bidirectional travel, door interlocking, door timing, and idle status using discrete logic and internal memory bits.

- Initialize the elevator to Floor 1 when no floor-position bit is active.
- Store a Floor 2 request while the elevator is at Floor 1.
- Permit upward motion only when the Door Closed condition is true and Door Open state is false.
- Prevent simultaneous Up and Down motor commands.
- Simulate upward travel for five seconds and update the stored floor position to Floor 2.
- Open the door at Floor 2 and keep it open for five seconds.
- Clear the door-open state and request after the door timer completes.
- Support visible downward travel logic from Floor 2 to Floor 1 using a five-second timer.
- Open the door at Floor 1 after downward arrival when Request_Active is present.
- Provide a no-active-sequence status bit when no request, motion, or door-open state is active.

3. Functional Requirements

- If Floor_1, Floor_2, and Floor_3 memory bits are all false, B3:0/0 Floor_1 shall latch.
- While at Floor 1, Floor 2 Button I:1/1 shall latch B3:0/3 Request_Active.
- Move_Up B3:0/4 shall latch when Request_Active, Floor_1, Door Closed I:1/3, and XIO Door_Open are all satisfied.
- Motor Up O:2/0 shall energize only when Move_Up is true and Move_Down is false.
- T4:1 shall time upward travel for five seconds.
- When T4:1/DN becomes true, Floor_1 shall clear, Floor_2 shall latch, and Move_Up shall clear.
- At Floor 2 with Request_Active true, Door_Open B3:0/6 shall latch.
- Door Open Solenoid O:2/2 shall energize while Door_Open is true.
- T4:0 shall time the door-open period for five seconds.
- When T4:0/DN becomes true, Door_Open and Request_Active shall clear.
- Move_Down B3:0/5 shall latch when Request_Active, Floor_2, Door Closed, and XIO Door_Open are satisfied.

- Motor Down O:2/1 shall energize only when Move_Down is true and Move_Up is false.
- T4:2 shall time downward travel for five seconds.
- When T4:2/DN becomes true, Floor_2 shall clear, Floor_1 shall latch, and Move_Down shall clear.
- At Floor 1 with Request_Active true, Door_Open shall latch.
- B3:0/7 shall energize when Request_Active, Move_Up, Move_Down, and Door_Open are all false.
- The report shall preserve the duplicate Elevator Running Lamp rungs 0010 and 0011 and shall not invent the missing Floor 1 request-generation rung.

4. System Components

Component	Identification	Function
PLC Platform	Allen-Bradley MicroLogix 1100	Executes the elevator sequence
Floor 2 Button	I:1/1	Visible request input for Floor 2
Door Closed	I:1/3	Motion permissive/interlock
Floor 1 Memory	B3:0/0	Stored Floor 1 position
Floor 2 Memory	B3:0/1	Stored Floor 2 position
Floor 3 Check	B3:0/2	Included in initialization rung
Request Active	B3:0/3	Stores an active elevator request
Move Up	B3:0/4	Upward motion state
Move Down	B3:0/5	Downward motion state
Door Open	B3:0/6	Door-open state
Idle / No Active Sequence	B3:0/7	Unlabeled status bit in screenshot
Motor Up	O:2/0	Upward motor command
Motor Down	O:2/1	Downward motor command
Door Open Solenoid	O:2/2	Door opening command
Elevator Running Lamp	O:2/3	Motion indication
Door Timer	T4:0	Times door-open period
Up Travel Timer	T4:1	Times upward travel
Down Travel Timer	T4:2	Times downward travel

5. I/O and Data Assignment

Physical I/O

Device	PLC Address	Type	Description
Floor 2 Button	I:1/1	Digital Input	Visible Floor 2 request input
Door Closed	I:1/3	Digital Input	Allows motion when door is closed
Motor Up	O:2/0	Digital Output	Upward travel command
Motor Down	O:2/1	Digital Output	Downward travel command
Door Open Solenoid	O:2/2	Digital Output	Door-open command
Elevator Running Lamp	O:2/3	Digital Output	Movement indication

Internal Memory

Function	Address	Purpose
Floor 1	B3:0/0	Stored Floor 1 state
Floor 2	B3:0/1	Stored Floor 2 state
Floor 3	B3:0/2	Checked only in initialization rung shown
Request Active	B3:0/3	Stored request state
Move Up	B3:0/4	Upward motion state
Move Down	B3:0/5	Downward motion state
Door Open	B3:0/6	Door-open state
Unlabeled status bit	B3:0/7	True when request/motion/door-open states are all inactive

Timer Assignment

Timer	Time Base	Preset	Duration	Function
T4:0	0.01 s	500	5.00 s	Door open
T4:1	0.01 s	500	5.00 s	Up travel
T4:2	0.01 s	500	5.00 s	Down travel

6. Control Strategy

I used internal memory bits to represent the stored floor position, active request, motion direction, and door-open state.

The Floor 2 request path begins only when Floor_1 is active. Once Request_Active is latched, the Move_Up state requires both the Door Closed input and a false Door_Open state. This separates the request from the motion command and introduces a basic door/motion interlock.

Motor Up and Motor Down each include an XIO of the opposite direction bit, preventing both physical direction outputs from being commanded simultaneously by the visible logic.

Travel is simulated with five-second TON timers. At timer completion, the current floor memory is cleared, the destination floor memory is latched, and the active motion state is cleared.

The door sequence is also state-based: arrival plus Request_Active latches Door_Open, the solenoid follows Door_Open, and a five-second door timer then clears Door_Open and Request_Active.

7. Engineering Design Decisions

Design Decision	Reason
Used internal floor-memory bits	Allowed the PLC to remember elevator position without physical floor sensors in the visible simulation
Used Request_Active bit	Separated request storage from movement outputs
Required Door Closed and XIO Door_Open before motion	Introduced a basic software door/motion interlock
Used separate Move Up and Move Down bits	Made direction state explicit
Added opposite-direction XIO interlocks	Prevented simultaneous Up/Down motor commands
Used five-second travel timers	Simulated elevator travel between floors
Used Door_Open latch plus five-second timer	Created a repeatable door-open sequence
Cleared Request_Active after door timing	Completed the visible Floor 2 request cycle
Preserved duplicate running-lamp rungs	Kept Version 1 faithful to the current screenshots
Did not invent a Floor 1 request rung	Maintained source-of-truth integrity because that rung is not visible

This Version 1 program is a simulation-oriented elevator sequence. It should not be interpreted as a real elevator safety controller, which would require certified hardware, redundant safety circuits, physical position feedback, door locks, limit devices, and applicable code compliance.

8. Ladder Logic Description

The current screenshots show eighteen functional rungs, Rung 0000 through Rung 0017, plus END at Rung 0018. All typed rungs below follow the approved single-line portfolio style.

8.1 Rung 0000 - Initialize Floor 1

-----[XIO B3:0/0]-----[XIO B3:0/1]-----[XIO B3:0/2]----- (OTL B3:0/0)-----
Floor_1 Floor_2 Floor_3 Floor_1

If none of the three floor-memory bits is active, Floor_1 is latched as the initial/default stored position.

8.2 Rung 0001 - Store Floor 2 Request

```
|----[ XIC B3:0/0 ]----[ XIC I:1/1 ]----- ( OTL B3:0/3 )----|
      Floor_1          Floor_2_Button          Request_Active
```

A Floor 2 request is stored only while the elevator is at Floor 1.

8.3 Rung 0002 - Latch Move Up

```
|----[ XIC B3:0/3 ]----[ XIC B3:0/0 ]----[ XIC I:1/3 ]----[ XIO B3:0/6 ]---- ( OTL B3:0/4 )----|
      Request_Active      Floor_1          Door_Closed      Door_Open          Move_Up
```

Upward motion is enabled only with an active request, Floor 1 position, Door Closed input true, and Door_Open false.

8.4 Rung 0003 - Motor Up Control

```
|----[ XIC B3:0/4 ]----[ XIO B3:0/5 ]----- ( OTE 0:2/0 )----|
      Move_Up          Move_Down          Motor_Up
```

Motor Up is energized only while Move_Up is active and Move_Down is inactive.

8.5 Rung 0004 - Up Travel Timer

```
|----[ XIC B3:0/4 ]-----[ TON T4:1 ]----|
      Move_Up          Travel_Timer  PRE: 500   Time Base: 0.01
```

T4:1 simulates five seconds of upward travel.

8.6 Rung 0005 - Floor 2 Arrival

```
|----[ XIC B3:0/4 ]----[ XIC T4:1/DN ]-----+---- ( OTU B3:0/0 )----|
      Move_Up          Travel_Timer_DN      |      Floor_1
                                           +---- ( OTL B3:0/1 )----|
                                           |      Floor_2
                                           +---- ( OTU B3:0/4 )----|
                                           Move_Up
```

At the end of upward travel, Floor_1 clears, Floor_2 latches, and Move_Up clears.

8.7 Rung 0006 - Open Door at Floor 2

```
|----[ XIC B3:0/1 ]----[ XIC B3:0/3 ]----- ( OTL B3:0/6 )----|
      Floor_2          Request_Active          Door_Open
```

At Floor 2 with Request_Active still true, Door_Open is latched.

8.8 Rung 0007 - Door Open Solenoid

```
|----[ XIC B3:0/6 ]----- ( OTE 0:2/2 )----|
      Door_Open          Door_Open_Solenoid
```

The door solenoid follows the Door_Open state.

8.9 Rung 0008 - Door Open Timer

```
|----[ XIC B3:0/6 ]-----[ TON T4:0 ]----|
      Door_Open              Open_Timer   PRE: 500   Time Base: 0.01
```

T4:0 holds the door-open sequence for five seconds.

8.10 Rung 0009 - Complete Door Cycle / Clear Request

```
|----[ XIC T4:0/DN ]-----+----( OTU B3:0/6 )----|
      Open_Timer_Done          |      Door_Open
                              +----( OTU B3:0/3 )----|
                              Request_Active
```

When the door timer completes, Door_Open and Request_Active are cleared.

8.11 Rung 0010 - Elevator Running Lamp

```
|----+----[ XIC B3:0/4 ]----+------( OTE 0:2/3 )----|
      |      Move_Up          |      Elevator_Running_Lamp
      +----[ XIC B3:0/5 ]----+
      Move_Down
```

The running lamp is energized during either upward or downward motion.

8.12 Rung 0011 - Duplicate Elevator Running Lamp

```
|----+----[ XIC B3:0/4 ]----+------( OTE 0:2/3 )----|
      |      Move_Up          |      Elevator_Running_Lamp
      +----[ XIC B3:0/5 ]----+
      Move_Down
```

The current screenshot repeats the same running-lamp logic on Rung 0011. I preserve this duplication exactly as implemented in Version 1.

8.13 Rung 0012 - Latch Move Down

```
|----[ XIC B3:0/3 ]----[ XIC B3:0/1 ]----[ XIC I:1/3 ]----[ XIO B3:0/6 ]----( OTL B3:0/5 )----|
      Request_Active      Floor_2          Door_Closed      Door_Open          Move_Down
```

Visible down-motion logic requires Request_Active, Floor 2, Door Closed, and Door_Open false. The separate rung that creates a new Floor 1 request is not shown.

8.14 Rung 0013 - Motor Down Control

```
|----[ XIC B3:0/5 ]----[ XIO B3:0/4 ]------( OTE 0:2/1 )----|
      Move_Down          Move_Up              Motor_Down
```

Motor Down is energized only while Move_Down is active and Move_Up is inactive.

8.15 Rung 0014 - Down Travel Timer

```
|----[ XIC B3:0/5 ]-----[ TON T4:2 ]----|
      Move_Down              Down_Time_Timer   PRE: 500   Time Base: 0.01
```

T4:2 simulates five seconds of downward travel.

8.16 Rung 0015 - Floor 1 Arrival

```

|----[ XIC B3:0/5 ]----[ XIC T4:2/DN ]-----+----( OTU B3:0/1 )----|
      Move_Down           Down_Timer_DN      |      Floor_2
                                           +----( OTL B3:0/0 )----|
                                           |      Floor_1
                                           +----( OTU B3:0/5 )----|
                                           Move_Down

```

At the end of downward travel, Floor_2 clears, Floor_1 latches, and Move_Down clears.

8.17 Rung 0016 - Open Door at Floor 1

```

|----[ XIC B3:0/0 ]----[ XIC B3:0/3 ]-----+----( OTL B3:0/6 )----|
      Floor_1           Request_Active      |      Door_Open

```

At Floor 1 with Request_Active true, the door-open state is latched.

8.18 Rung 0017 - Idle / No Active Sequence Status

```

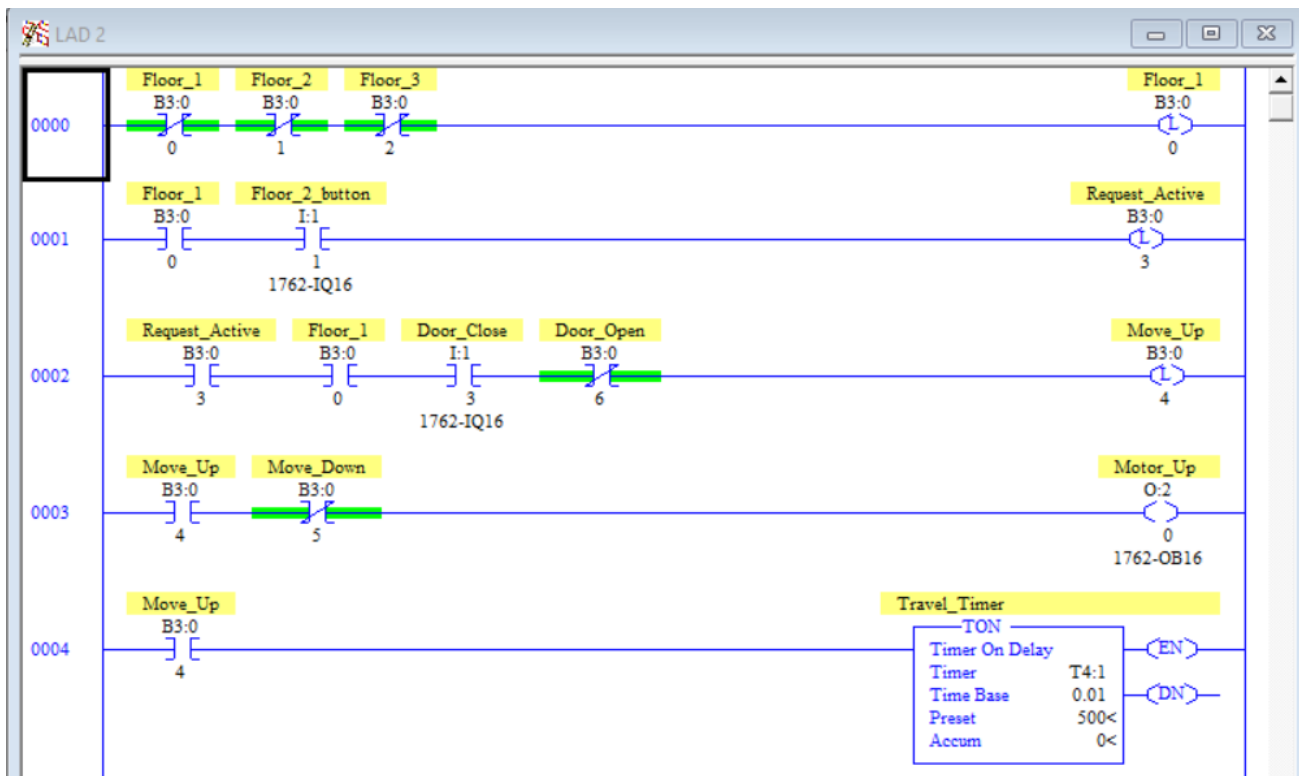
|----[ XIO B3:0/3 ]----[ XIO B3:0/4 ]----[ XIO B3:0/5 ]----[ XIO B3:0/6 ]----( OTE B3:0/7 )----|
      Request_Active      Move_Up           Move_Down           Door_Open           B3:0/7

```

B3:0/7 becomes true when no request, movement, or door-open state is active. Because the screenshot does not show a symbolic label, I describe it functionally rather than inventing a tag name.

8.19 Actual RSLogix Implementation

The four figures below are the current RSLogix screenshots used as the technical source of truth for this report.



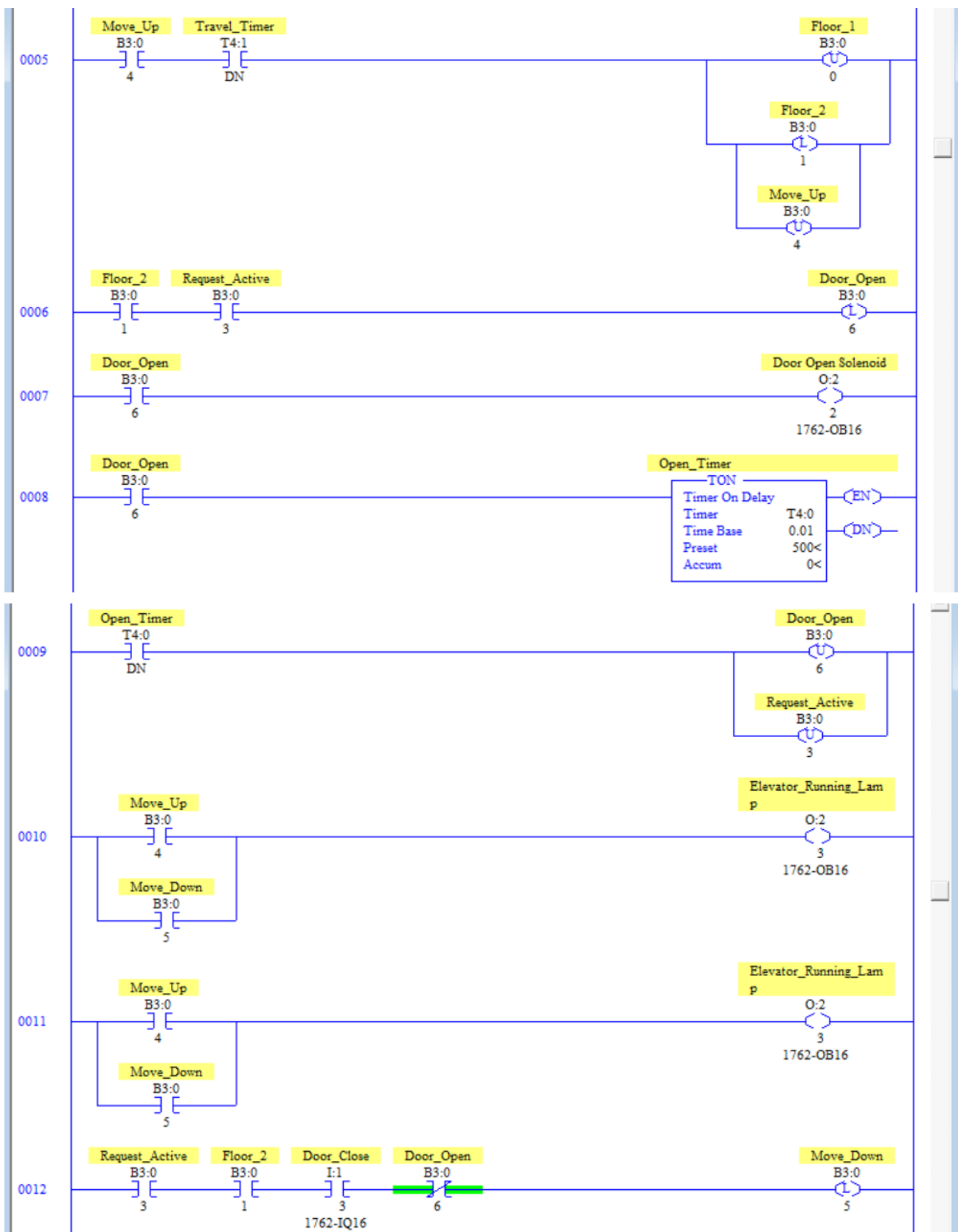


Figure P9.1 - RSLogix Elevator Logic

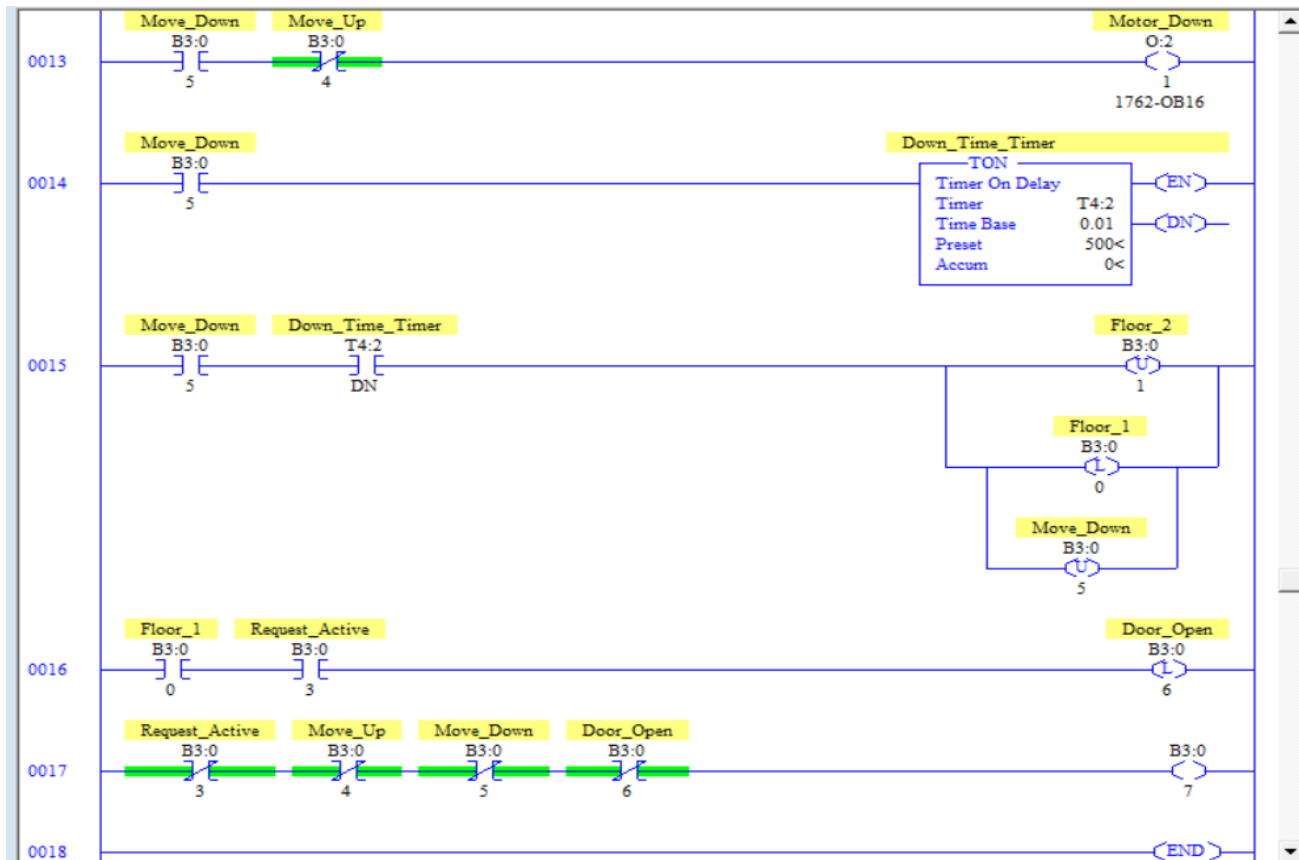


Figure P9.1 - RSLogix Elevator Logic

9. Sequence of Operation

Step 1 - Initialization: If no floor-memory bit is active, Floor_1 B3:0/0 is latched.

Step 2 - Floor 2 Request: At Floor 1, Floor 2 Button I:1/1 latches Request_Active.

Step 3 - Up Motion Permissive: Door Closed must be true and Door_Open false before Move_Up latches.

Step 4 - Upward Travel: Motor Up energizes and T4:1 times for five seconds.

Step 5 - Floor 2 Arrival: Floor_1 clears, Floor_2 latches, and Move_Up clears.

Step 6 - Door Open at Floor 2: Floor_2 plus Request_Active latches Door_Open.

Step 7 - Door Open Period: Door solenoid energizes and T4:0 times for five seconds.

Step 8 - Request Complete: T4:0/DN clears Door_Open and Request_Active.

Step 9 - Idle: With request, motion, and door-open states false, B3:0/7 becomes true.

Visible Down Sequence: When Request_Active is present at Floor 2 with Door Closed and Door_Open false, Move_Down can latch.

Downward Travel: Motor Down energizes and T4:2 times for five seconds.

Floor 1 Arrival: Floor_2 clears, Floor_1 latches, and Move_Down clears.

Door Open at Floor 1: Floor_1 plus Request_Active latches Door_Open.

Source Limitation: The screenshot does not show the separate rung that creates a new Floor 1 request, so no specific Floor 1 button/address is claimed.

10. Testing & Validation

The current ladder screenshots represent the final tested Version 1 implementation. Tests below are derived only from behavior directly supported by the visible logic and are recorded as PASS.

Test ID	Test Action / Condition	Expected Result	Observed Result	Status
T-01	No floor-memory bit active	Floor_1 latches	Floor_1 initialized	PASS
T-02	At Floor 1, activate Floor 2 Button	Request_Active latches	Request stored	PASS
T-03	Request active but Door Closed false	Move_Up does not latch	Motion remained inhibited	PASS
T-04	Request active, Floor 1, Door Closed true, Door_Open false	Move_Up latches; Motor Up ON	Up motion started	PASS
T-05	Up travel reaches 5 s	Floor_1 clears; Floor_2 latches; Move_Up clears	Floor state updated	PASS
T-06	Floor 2 + Request_Active	Door_Open latches; solenoid ON	Door sequence started	PASS
T-07	Door-open timer reaches 5 s	Door_Open and Request_Active clear	Door/request cycle completed	PASS
T-08	Move_Up active while Move_Down false	Motor Up ON; Motor Down OFF	Direction interlock correct	PASS
T-09	With Request_Active established at Floor 2 and door permissive satisfied	Move_Down latches; Motor Down ON	Visible down-motion logic operated	PASS
T-10	Down travel reaches 5 s	Floor_2 clears; Floor_1 latches; Move_Down clears	Floor 1 arrival completed	PASS
T-11	Floor 1 + Request_Active	Door_Open latches	Floor 1 door-open logic operated	PASS
T-12	No request, no motion, door closed/not open	B3:0/7 true	Idle/no-active-sequence status asserted	PASS
T-13	Move Up active	Elevator Running Lamp O:2/3 ON	Running indication active	PASS
T-14	Move Down active	Elevator Running Lamp O:2/3 ON	Running indication active	PASS

Validation Summary

Testing confirmed the fully visible Floor 1-to-Floor 2 request sequence, door/motion permissive behavior, upward travel, floor-state update, door timing, request clearing, motor direction interlocks, the visible downward motion and Floor 1 arrival logic, and the idle/status condition.

The Floor 1 request-generation rung is not visible, so the report does not claim a specific Floor 1 request input test. Instead, the down sequence is validated from the point where Request_Active is already established at Floor 2.

All fourteen defined source-supported Version 1 functional tests passed. This is functional PLC project validation, not real elevator commissioning or safety certification.

11. Results

The completed Elevator Control System successfully demonstrated stored floor position, request handling, door-permissive motion, bidirectional motor control, timer-simulated travel, floor-state transitions, door timing, and idle-status logic.

The fully visible Floor 1-to-Floor 2 sequence operated from Floor 2 request through upward travel, Floor 2 arrival, door opening, five-second door timing, and request completion.

The current screenshots also contain working downward travel and Floor 1 arrival logic when Request_Active is present at Floor 2. Motor Up and Motor Down include mutual interlocks, and the running lamp follows either movement state.

All fourteen defined source-supported functional tests passed.

12. Challenges & Troubleshooting

A useful troubleshooting path for this project is: Floor Memory -> Request_Active -> Door Permissive -> Move Direction -> Motor Output -> Travel Timer -> Destination Floor -> Door State -> Door Timer -> Request Clear / Idle.

Symptom	Possible Cause	Diagnostic Check	Corrective Action
Elevator does not initialize to Floor 1	One floor-memory bit remains active	Monitor B3:0/0, /1, /2	Verify Rung 0000 conditions
Floor 2 request is not stored	Not at Floor 1 or I:1/1 not true	Monitor B3:0/0, I:1/1, B3:0/3	Verify Rung 0001
Up motion does not start	Door Closed false, Door_Open true, request/floor state incorrect	Monitor I:1/3, B3:0/6, B3:0/3, B3:0/0	Verify Rung 0002
Both directions appear possible	Direction-state logic incorrect	Monitor B3:0/4 and B3:0/5	Verify opposite-direction XIO interlocks
Floor 2 never becomes active	T4:1 does not reach Done	Monitor T4:1.ACC and T4:1/DN	Verify up timer and Rung 0005
Door does not open at Floor 2	Floor_2 or Request_Active false	Monitor B3:0/1 and B3:0/3	Verify Rung 0006
Door never completes	T4:0 not timing/done	Monitor B3:0/6 and T4:0	Verify door timer and Rung 0009
Down motion does not start	Request_Active absent or door permissive not satisfied	Monitor B3:0/3, B3:0/1, I:1/3, B3:0/6	Verify visible Rung 0012 conditions
Floor 1 never becomes active after down travel	T4:2 does not reach Done	Monitor T4:2.ACC and T4:2/DN	Verify Rung 0015
Idle/status bit not true	A request, movement, or Door_Open state remains active	Monitor B3:0/3 through B3:0/6	Verify Rung 0017

This project reinforced that sequence troubleshooting starts with the stored state. A motion or door problem is often caused by an earlier state or permissive not being satisfied rather than by the final output rung itself.

13. Lessons Learned

This project helped me understand how internal memory bits can represent both physical-position state and sequence state in a simulated machine.

I learned to separate a stored request from the motion command and to apply door conditions before allowing movement.

The opposite-direction interlocks demonstrated how mutually exclusive outputs can be enforced in ladder logic.

The arrival rungs showed how a timer Done condition can update multiple states together: clear the source floor, latch the destination floor, and clear the movement command.

The project also showed why documentation must follow the actual ladder. The duplicate running-lamp rung, Floor_3 initialization check, unlabeled B3:0/7, and missing visible Floor 1 request rung are retained as part of the Version 1 record rather than being silently rewritten.

14. Industrial Relevance

Elevator-style motion sequencing is useful as a learning model for industrial hoists, lifts, vertical transfer units, indexing systems, and other bidirectional position-based machines.

The Version 1 project demonstrates general automation concepts such as request storage, position state, door/motion permissives, direction interlocking, timed travel, arrival transitions, and door sequencing.

It is not a real elevator controller. Actual passenger or freight elevators require certified safety systems, redundant braking and position devices, door locks, overspeed protection, emergency circuits, code-compliant hardware, inspections, and many other safety functions that are outside the scope of this PLC exercise.

The engineering value of this project is therefore the sequencing and interlocking method, not direct applicability to a real elevator installation.

15. Transition to Industrial Version (Version 2)

Version 2 can retain the request/motion/door sequencing concept while replacing simulation assumptions with stronger industrial state management, diagnostics, permissives, and fault recovery.

Version 2 Improvement	Engineering Benefit
Explicit System Ready / Run architecture	Defines when elevator logic is permitted to operate
Complete floor-request handling	Provides explicit request logic for every supported floor
Physical floor sensors	Replaces timer-only position assumption with actual feedback
Validated floor-state logic	Detects impossible or conflicting floor indications
Door-cycle active state	Separates door sequence from simple Door_Open latch
Door closed/open feedback validation	Verifies physical door condition
Motion permissive	Centralizes conditions required before movement
Direction-conflict detection	Detects simultaneous Up/Down command conditions
Travel timeout fault	Detects failure to reach a destination
Door timeout fault	Detects incomplete door operation
Fault latch and controlled reset	Provides defined abnormal-condition recovery
Auto/Manual modes	Supports normal operation and maintenance control
Restart prevention	Prevents unexpected movement after safety/permissive restoration
HMI status and diagnostics	Displays floor, request, mode, motion, door, timers, and faults

A stronger Version 2 architecture could become: Request -> Validate Current Floor -> Motion Permissive -> Direction Command -> Physical Travel Feedback -> Arrival -> Door Cycle -> Request Complete -> Idle, with fault detection and controlled recovery around every stage.

The Version 2 design should preserve the clear request/motion/door sequence while removing the ambiguities and simulation limitations visible in Version 1.

16. Project Summary

The Elevator Control System introduced request-based bidirectional motion, stored floor position, door interlocking, timer-simulated travel, door sequencing, and idle-status logic.

The fully visible Floor 1-to-Floor 2 sequence begins with Floor 2 Button I:1/1, stores Request_Active, checks Door Closed, latches Move_Up, energizes Motor Up, times five seconds of travel, updates the stored floor position, opens the door for five seconds, and then clears the request.

The screenshots also contain the visible Move Down, Motor Down, five-second down travel, Floor 1 arrival, and Floor 1 door-open logic. Because the separate Floor 1 request-generation rung is not shown, the report does not invent its input or implementation.

All fourteen defined source-supported functional tests passed. The project strengthened my understanding of state memory, request handling, motion permissives, direction interlocking, timed transitions, door sequencing, and faithful source-based engineering documentation.